

POLITECNICO DI MILANO

Compressed Sensing using Generative Models

Matteo Forlivesi
Renato Gallicola

1 SUMMARY

1.1 INTRODUCTION

This experiment aims to evaluate the performance of compressed sensing using two models on the MNIST dataset: a Variational Auto-encoder (VAE) and a Generative Adversarial Network (GAN). The focus is on image reconstruction from compressed measurements. The MNIST dataset consists of 60,000 training images and 10,000 test images of handwritten digits, each of size 28x28 pixels.

1.2 MODELS AND METHODOLOGY

We employed two approaches to reconstruct the images from compressed measurements:

1. **Variational Auto-encoder (VAE):** This model is designed to generate images similar to those in the training set. The VAE compresses the input image into a lower-dimensional latent space and then reconstructs the image from this representation.
2. **Generative Adversarial Network (GAN):** This model is composed of two competing networks, a generative one and a discriminating one. They are trained to minimize their losses:
 - The generator tries to generate images from a random latent vector.
 - The discriminator tries to discern between real images and generated ones.

For both models, we used a random Gaussian matrix for initial measurements. The compression levels varied from 10 to 750 measurements per image, out of the 784 pixels of a full image, allowing us to observe the reconstruction quality across different levels of compression.

1.3 ALGORITHM EXPLANATION

This section will explain the algorithm utilizing a generative model for compressed sensing.

Enhancements include training for 1000 epochs and 10 random restarts to improve convergence and error minimization.

1.4 THE MNIST DATASET

The MNIST (Modified National Institute of Standards and Technology) dataset is a large collection of handwritten digits commonly used for training and evaluating image processing systems. It contains 60,000 training images and 10,000 test images, each being a gray scale image of size 28×28 pixels. Each image is labeled with a digit from 0 to 9, providing a diverse set of handwriting styles for developing robust digit recognition systems.

- **Size:** 60,000 training images and 10,000 test images.
- **Resolution:** Each image is 28×28 pixels, resulting in 784 features.
- **Classes:** 10 classes, representing digits 0 through 9.
- **Grayscale:** Pixel values range from 0 (black) to 255 (white).

The MNIST images were vectorized, resulting in 784-dimensional input vectors. Each pixel value was normalized to the range $[0, 1]$.

1.5 RESULTS

The reconstruction performance was evaluated as the squared error between the original and the reconstructed image, divided by the number of pixels, averaged over ten test images, one for each digit class. The following observations were made:

1. **Performance with few measurements:** both models are far more accurate than LASSO when measurements are scarce. The VAE with a latent dimension of 30 obtains with 75 measurements the accuracy that LASSO reaches with 400, and with 25 measurements its error is 4.2 times lower than the LASSO one.
2. **Saturation:** the accuracy of both models stops improving after roughly 200 measurements, because the reconstruction is constrained to the range of the generator. From 500 measurements onwards LASSO becomes the most accurate method.
3. **Comparison between the two models:** the VAE with a latent dimension of 30 is the most accurate of the five methods at seven of the ten budgets tested, and its recovery is about 38 times faster than the DCGAN one.

1.6 CONCLUSION

The experiment demonstrated the efficacy of using deep learning models, particularly VAEs and GANs, for image reconstruction from compressed measurements. These models significantly reduce the reconstruction error compared to traditional sparse recovery methods when the number of measurements is small, while the traditional method remains preferable when measurements are abundant. The findings highlight the potential of integrating compressed sensing techniques with deep learning to enhance image reconstruction quality in resource-constrained environments.

These results provide a promising direction for further research, particularly in optimizing measurement matrices and exploring other deep learning architectures for compressed sensing applications.

2 INTRODUCTION

Compressed sensing is a signal processing technique that enables the reconstruction of a signal from a small number of measurements. Given the measurement y , the measurements matrix $A \in \mathbb{R}^{m \times n}$ and the noise $\eta \in \mathbb{R}^m$, the goal is to estimate the vector $x^* \in \mathbb{R}^n$ such that:

$$y = Ax^* + \eta \quad (2.1)$$

Since $m \ll n$, this system is underdetermined, then it is not possible to find a unique solution. The NyquistShannon sampling theorem requires a certain amount of samples to perfectly reconstruct a signal, so compressed sensing exploits some features of the signal to address this problem in a context where the conditions of the aforementioned theorem are not met.

The standard version of the algorithm assumes the signal to be k -sparse, meaning that it has only k coefficients different from zero when represented in some domain. This assumption allows the signal retrieval through the use of algorithms for solving minimization problems, such as LASSO [11] for the L^1 norm minimization, under conditions on A established by the classical compressed sensing literature [2, 3].

Following Bora et al. [1], this experiment shifts from sparsity to structures derived from generative models, with the aim of assessing the goodness of these models when employed in compressed sensing. They feature a generator that maps vectors from a low dimensional to a high dimensional space, so from $z \in \mathbb{R}^k$ to $G(z) \in \mathbb{R}^n$. By training the generator, with high probability it will output vectors similar to the ones belonging to the dataset it has been trained on.

The idea behind the use of generative models in the context of compressed sensing is that we expect the vector x^* to be close to some $G(z)$, which can be found with an optimization algorithm based on gradient descent, that will be later discussed in detail.

3 MODELS

3.1 DCGAN

Generative Adversarial Networks (GANs) are a class of machine learning frameworks introduced by Goodfellow et al. in 2014 [4]. GANs have revolutionized the field of generative modeling, allowing for the creation of highly realistic synthetic data. This section will explore the fundamental concepts, structure, and applications of Deep Convolutional Generative Adversarial Networks (DCGANs).

3.1.1 INTRODUCTION TO DCGANS

DCGANs consist of two neural networks: the generator and the discriminator, which compete to minimize their respective loss functions. The generator aims to create data that is indistinguishable from real data, while the discriminator attempts to differentiate between real and generated data. This adversarial process continues until the generator produces data that the discriminator cannot reliably distinguish from real data. DCGANs leverage convolutional neural networks (CNNs) to improve the quality of generated images.

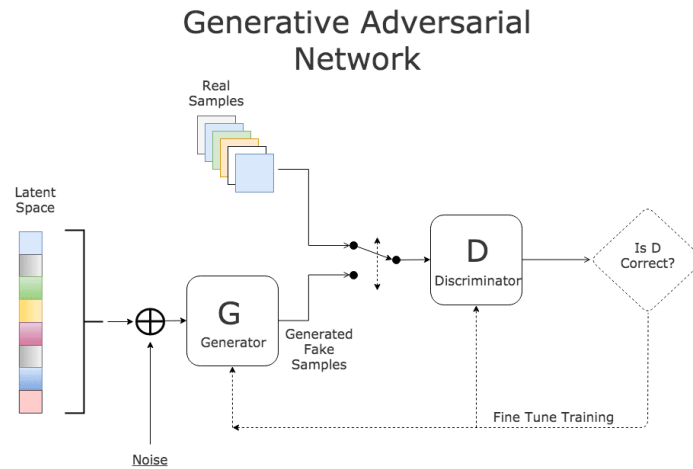


Figure 3.1: Structure of a Generative Adversarial Network, picture from [5]

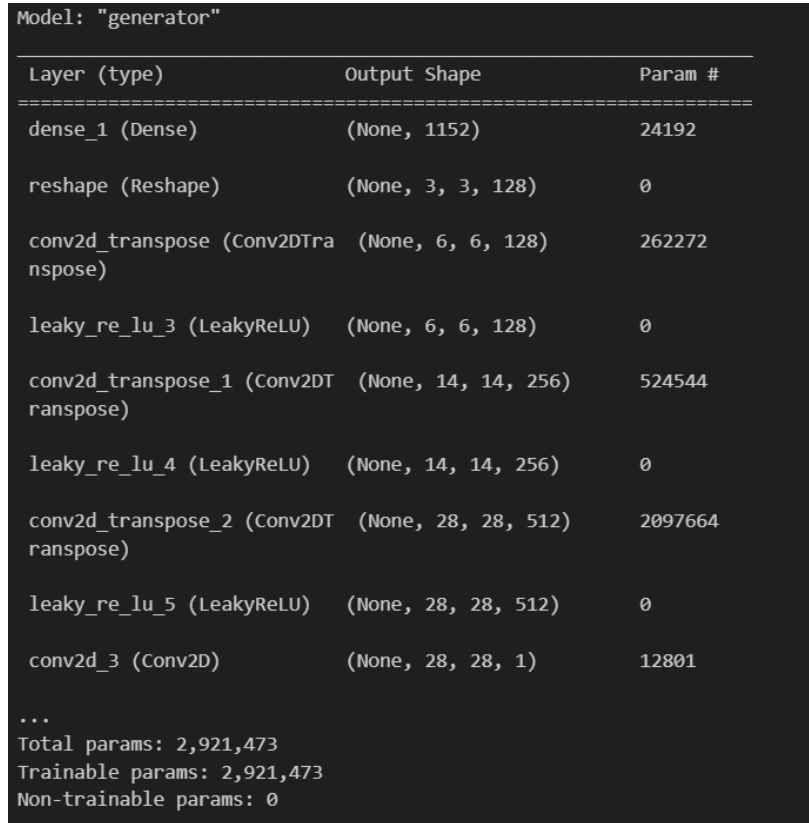
3.1.2 COMPONENTS OF DCGANS

The two primary components of DCGANs are:

GENERATOR

The generator, G , is a convolutional neural network that takes random noise, z , from a latent space and transforms it into a synthetic data sample, $G(z)$. The objective of the generator is to produce data that is as close as possible to the real data distribution. The generator's loss function is designed to minimize the likelihood of the discriminator correctly identifying the generated data as fake.

The actual structure of the generator, inspired by the standard DCGAN presented in [9]:



```
Model: "generator"
Layer (type)                Output Shape                Param #
-----
dense_1 (Dense)              (None, 1152)                24192
reshape (Reshape)            (None, 3, 3, 128)           0
conv2d_transpose (Conv2DTra (None, 6, 6, 128)           262272
nspose)
leaky_re_lu_3 (LeakyReLU)    (None, 6, 6, 128)           0
conv2d_transpose_1 (Conv2DT (None, 14, 14, 256)          524544
ranspose)
leaky_re_lu_4 (LeakyReLU)    (None, 14, 14, 256)         0
conv2d_transpose_2 (Conv2DT (None, 28, 28, 512)          2097664
ranspose)
leaky_re_lu_5 (LeakyReLU)    (None, 28, 28, 512)         0
conv2d_3 (Conv2D)            (None, 28, 28, 1)           12801
...
Total params: 2,921,473
Trainable params: 2,921,473
Non-trainable params: 0
```

Figure 3.2: Structure of a Deep Convolutional Generative Adversarial Network Generator

DISCRIMINATOR

The discriminator, D , is another convolutional neural network that evaluates data and assigns a probability that a given sample comes from the real data distribution rather than the generator. The discriminator's loss function aims to maximize the likelihood of correctly classifying real and fake data.

The actual structure of the discriminator, inspired by the standard DCGAN presented in [9]:

Model: "Discriminator"		
Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 14, 14, 64)	1088
leaky_re_lu (LeakyReLU)	(None, 14, 14, 64)	0
conv2d_1 (Conv2D)	(None, 7, 7, 128)	131200
leaky_re_lu_1 (LeakyReLU)	(None, 7, 7, 128)	0
conv2d_2 (Conv2D)	(None, 4, 4, 128)	262272
leaky_re_lu_2 (LeakyReLU)	(None, 4, 4, 128)	0
flatten (Flatten)	(None, 2048)	0
dropout (Dropout)	(None, 2048)	0
dense (Dense)	(None, 1)	2049
=====		
Total params: 396,609		
Trainable params: 396,609		
Non-trainable params: 0		

Figure 3.3: Structure of a Deep Convolutional Generative Adversarial Network Discriminator

3.1.3 TRAINING PROCESS

The training process for DCGANs involves optimizing the following value function, $V(G, D)$, which is a minimax game between G and D :

$$\min_G \max_D V(G, D) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))] \quad (3.1)$$

The training of DCGANs involves alternating between updating the discriminator and the generator. The discriminator is trained to maximize its classification accuracy, while the generator is trained to deceive the discriminator. This iterative process can be summarized in the following steps:

One detail of our implementation is worth mentioning: the labels given to the discriminator are perturbed by a small amount of uniform noise, drawn in $[0, 0.05]$. This keeps the discriminator from saturating early in training, which

would leave the generator without a useful gradient.

The procedure can be summarized in the following steps:

One detail of our implementation is worth mentioning: the labels given to the discriminator are perturbed by a small amount of uniform noise, drawn in $[0, 0.05]$. This keeps the discriminator from saturating early in training, which would leave the generator without a useful gradient.

The procedure can be summarized in the following steps:

1. Sample a batch of real data and a batch of random noise.
2. Update the discriminator by minimizing its classification loss on both real and generated data, which corresponds to maximizing $V(G, D)$.
3. Sample a new batch of random noise.
4. Update the generator by minimizing its loss function, which depends on the discriminator’s performance.

3.1.4 TRAINING CONFIGURATION

Both networks are trained on the full MNIST collection, obtained by concatenating the training and test splits into 70,000 images, since no labels are involved and the model is never evaluated on held out classes. Training runs for 50 epochs with a batch size of 32, using one Adam optimizer for each network with a learning rate of 0.0001 and a binary cross-entropy loss. As for the VAE, two versions were trained, one with a latent vector of dimension 20 and one of dimension 30.

3.1.5 TRAINING CONFIGURATION

Both networks are trained on the full MNIST collection, obtained by concatenating the training and test splits into 70,000 images, since no labels are involved and the model is never evaluated on held out classes. Training runs for 50 epochs with a batch size of 32, using one Adam optimizer for each network with a learning rate of 0.0001 and a binary cross-entropy loss. As for the VAE, two versions were trained, one with a latent vector of dimension 20 and one of dimension 30.

3.2 VAE

3.2.1 INTRODUCTION TO VAE

The **Variational Auto-encoder** (VAE) [7] is an artificial neural network whose encoder and decoder are connected through a probabilistic latent space, corresponding to the parameters of a variational distribution.

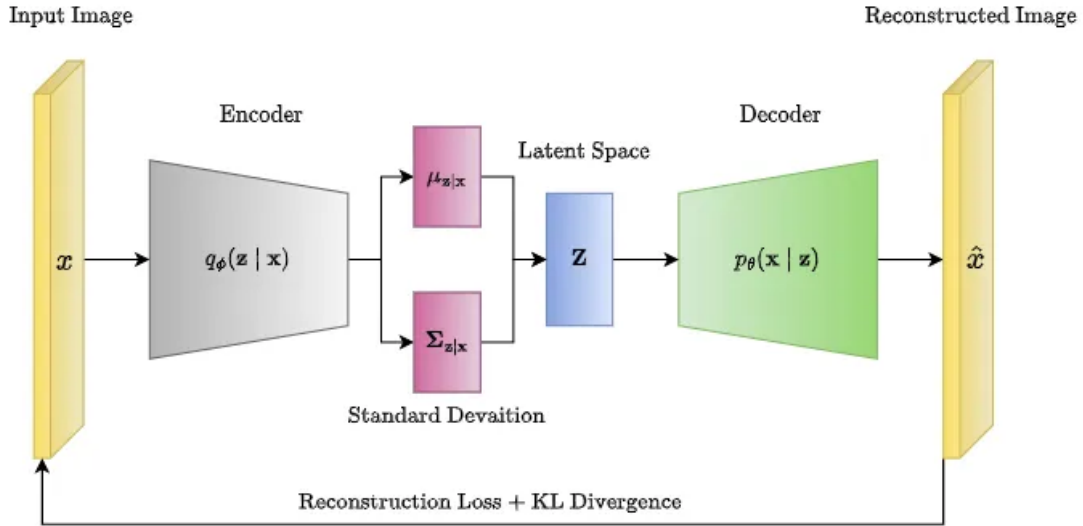


Figure 3.4: Structure of a Variational Auto-encoder, picture from [10]

The encoder receives as input a tensor representing an image extracted from a dataset, and it maps it into a distribution within the latent space, parametrized by a mean vector μ and a log-variance vector $\log \sigma^2$. The network predicts the logarithm of the variance instead of the standard deviation, so that its output is unconstrained in sign while σ remains positive.

The decoder, instead, maps from the latent space back to the input space.

The VAE's parameters are optimized via gradient descent, however the sampling of the latent variable z introduces stochasticity, making the gradient of the loss with respect to the model parameters non-differentiable. A technique known as **Reparameterization Trick** is employed to address this problem, which expresses the sampling operation in a way that separates the deterministic and stochastic parts, enabling gradient flow through the deterministic part. So instead of directly sampling z as $z \sim N(\mu, \sigma^2)$, which is non-differentiable

with respect to μ and σ , it sample an auxiliary variable ϵ from a standard normal distribution $N(0, I)$, and then it expresses z as $z = \mu + \sigma \cdot \epsilon$, which is now a differentiable function of μ and σ .

3.2.2 COMPONENTS OF VAE

The two primary components of the VAE are:

ENCODER

The encoders architecture consists of four 2D convolution layers, one flatten layer and one dense layer. The input received by this module, which is a 1-channel 28×28 image, passes through the convolutional layers – using 32, 64, 64 and 64 filters respectively – to extract features, then the feature maps is flattened and its dimensionality is reduced using a dense layer; finally the encoder outputs the mean and the log-variance of the latent space distribution and samples the latent vector z .

Model: "encoder"			
Layer (type)	Output Shape	Param #	Connected to
encoder_input (InputLayer)	(None, 28, 28, 1)	0	[]
conv2d (Conv2D)	(None, 28, 28, 32)	320	['encoder_input[0][0]']
conv2d_1 (Conv2D)	(None, 14, 14, 64)	18496	['conv2d[0][0]']
conv2d_2 (Conv2D)	(None, 14, 14, 64)	36928	['conv2d_1[0][0]']
conv2d_3 (Conv2D)	(None, 14, 14, 64)	36928	['conv2d_2[0][0]']
flatten (Flatten)	(None, 12544)	0	['conv2d_3[0][0]']
dense (Dense)	(None, 32)	401440	['flatten[0][0]']
z_mu (Dense)	(None, 2)	66	['dense[0][0]']
z_sigma (Dense)	(None, 2)	66	['dense[0][0]']
z (Lambda)	(None, 2)	0	['z_mu[0][0]', 'z_sigma[0][0]']
Total params: 494,244			
Trainable params: 494,244			
Non-trainable params: 0			

Figure 3.5: Structure of a VAE Encoder

DECODER

The decoders architecture consists of a dense layer, a reshape layer and two 2D transposed convolution layers. The input of the module is the latent vector that, through the dense and reshape layers, it is transformed back into a tensor, matching the shape of the last convolution layer of the encoder; finally the convolution layers generate the reconstructed image with the same number of channels as the original input image.

Model: "decoder"		
Layer (type)	Output Shape	Param #
decoder_input (InputLayer)	[(None, 2)]	0
dense_1 (Dense)	(None, 12544)	37632
reshape (Reshape)	(None, 14, 14, 64)	0
conv2d_transpose (Conv2DTranspose)	(None, 28, 28, 32)	18464
decoder_output (Conv2DTranspose)	(None, 28, 28, 1)	289
=====		
Total params: 56,385		
Trainable params: 56,385		
Non-trainable params: 0		

Figure 3.6: Structure of VAE Decoder

3.2.3 TRAINING PROCESS

The optimization of VAE is achieved through the maximization of the **Evidence Lower Bound** (ELBO), consisting of two terms: the **Reconstruction Loss** and the **Regularization Term** (KullbackLeibler Divergence).

Given the input data x , the latent variable z , the encoder network $q_\phi(z|x)$, the decoder network $p_\theta(x|z)$, the prior distribution over the latent variables $p(z)$, the reconstruction loss $\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)]$, and the KL divergence $KL(q_\phi(z|x) || p(z))$, the loss is computed as:

$$L(\theta, \phi; x) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - KL(q_\phi(z|x) || p(z)) \quad (3.2)$$

The VAE is trained with a custom loop, consisting of three main steps: **forward pass**, **loss calculation** and **backward pass**. During the forward pass, the input data is passed through the encoder to get the mean and log variance of the latent distribution, then the latent vector z is sampled using the

reparametrization trick, and finally z is passed through the decoder to get the reconstructed data. The loss calculation step computes the reconstruction loss between the input data and the reconstructed data, the KL divergence, and then sums them up to compute the total loss. Finally, in the backward pass the total loss is backpropagated to update the parameters of the encoder and decoder.

The model is trained for up to 100 epochs using a batch size of 100 and an Adam optimizer [6] with learning rate of 0.001, with early stopping monitoring the loss.

To test the goodness of the model in the context of compress sensing, two versions of the VAE have been generated: one with the dimension of the latent vector equal to 20, and the other equal to 30. After using both of them, its possible to conclude that they lead to good results, as shown in the Results section.

3.3 MODEL RESULTS

The following graph shows the output mean vector produced by the encoder with the test set provided by the MNIST dataset. For the sake of simplicity, this result is obtained with a latent dimension equal to 2.

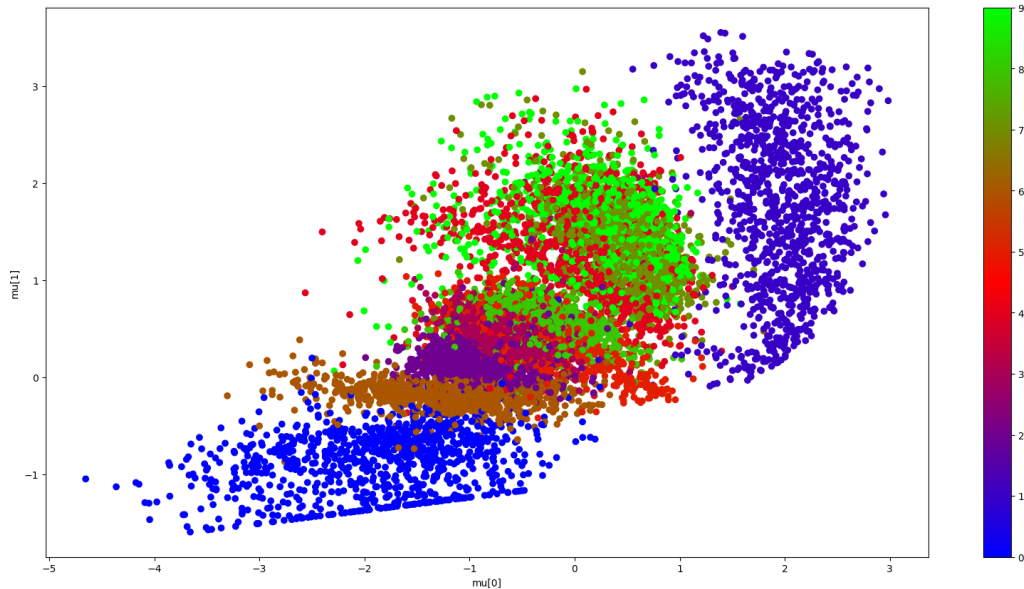


Figure 3.7: Plotted output of the encoder, each color represents a specific digit

By observing the data in the graph, we can predict the behaviour of the decoder. Infact, if we assume it receives as input the vector $[-2, -1]$, we expect the output to depict with good precision the number 0, as represented by the blue color in the lateral colorbar. The image reconstruction is shown in the image below.

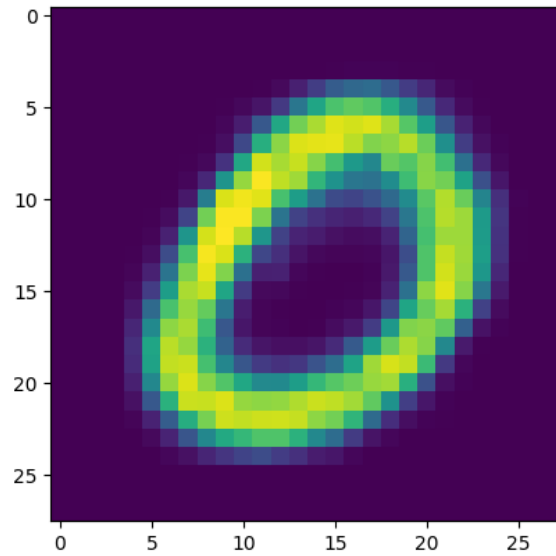


Figure 3.8: Output of the decoder for the input vector $[-2, -1]$

4 ALGORITHM EXPLANATION

Generative models offer a powerful approach for compressed sensing by leveraging the structure of the data distribution.

Independently from the above mentioned models, a generative model is defined by a deterministic function $G : \mathbb{R}^k \rightarrow \mathbb{R}^n$ and a distribution P_Z over $z \in \mathbb{R}^k$. The model maps a low-dimensional latent vector z (decided by us as the latent dimension) to the high-dimensional sample space (the final generated image). The compressed sensing task involves finding a vector z in the latent space such that the corresponding generated sample $G(z)$ matches the observed measurements y .

The measurement matrix A is drawn at random, with entries sampled independently from a Gaussian distribution $N(0, 1/m)$. With this scaling the operator preserves norms in expectation, since $\mathbb{E}\|Ax\|^2 = \|x\|^2$ for every x , and the magnitude of the measurement error remains comparable across different values of m .

The objective function to be minimized is:

$$\text{loss}(z) = \|AG(z) - y\|^2. \quad (4.1)$$

Bora et al. [1] also consider a regularized version of this objective, obtained by adding a term $\lambda\|z\|^2$ that keeps the solution inside the region where the prior P_Z places most of its mass. In our experiments we set $\lambda = 0$, so that the reconstruction depends on the generator alone.

Optimization is performed using gradient descent on z with the Adam algorithm [6] for optimizing descent. If G is differentiable, gradients of the loss with respect to z can be computed using back-propagation. The process involves iteratively updating z to minimize the measurement error. Upon convergence to $\hat{z}$, the reconstruction of x^* is given by $G(\hat{z})$.

The algorithm can be summarized as follows:

1. Initialize z by sampling from the distribution P_Z , which for both models is the standard normal $N(0, I)$ used during training.
2. Compute the loss: $\text{loss}(z) = \|AG(z) - y\|^2$.
3. Use gradient descent to update z to minimize the loss.
4. Repeat until convergence.

5. The reconstructed signal is $\hat{x} = G(\hat{z})$.

This method leverages the data prior encoded in the generative model to achieve high-quality reconstructions with fewer measurements compared to traditional compressed sensing methods.

A theoretical justification is given in [1]. If G is L -Lipschitz, a number of random Gaussian measurements of the order of $k \log L$ is sufficient to guarantee that the reconstruction error is close to the smallest error achievable by any vector in the range of G . The number of measurements required therefore scales with the latent dimension k and not with the ambient dimension n , which is the reason why so few measurements are enough.

In order to improve convergence we performed a thousand gradient steps for each of ten random restarts, and we kept the attempt that presented the smallest measurement error at the end of the approximation. These steps are not training epochs: the weights of G are frozen and the only variable that gets updated is the latent vector z .

5 THE MNIST DATASET

The MNIST (Modified National Institute of Standards and Technology) dataset [8] is a large collection of handwritten digits commonly used for training various image processing systems. It serves as a benchmark for evaluating the performance of machine learning algorithms. The dataset contains 60,000 training images and 10,000 test images, each of which is a gray scale image of size 28×28 pixels.

Each image in the MNIST dataset is labeled with the digit it represents, ranging from 0 to 9. The dataset is particularly valuable because it provides a diverse set of handwriting styles, allowing for the development of robust digit recognition systems.

- **Size:** The training set consists of 60,000 images, and the test set consists of 10,000 images.
- **Resolution:** Each image is 28×28 pixels, making a total of 784 features.
- **Classes:** There are 10 classes, corresponding to the digits 0 through 9.
- **Grayscale:** Images are in grayscale, with pixel values ranging from 0 (black) to 255 (white).

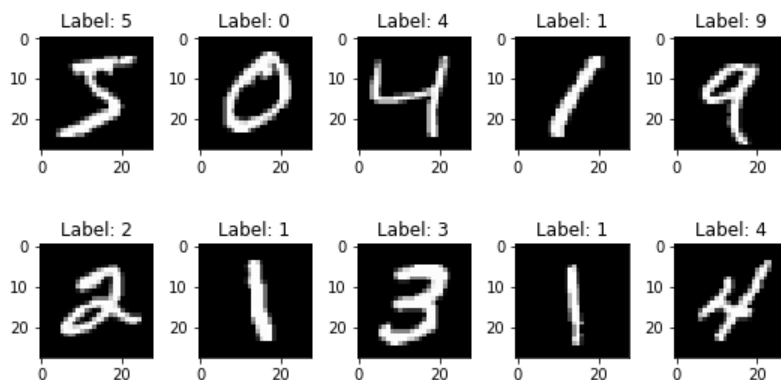


Figure 5.1: Sample of images from the MNIST dataset

6 RESULTS

We created two versions of both VAE and DCGAN using different lengths for the latent representation vector z . In one case, we used a latent vector of dimension 20, and in the other, a dimension of 30. Together with the LASSO baseline this gives five methods to compare.

6.1 EXPERIMENTAL PROTOCOL

Every method is evaluated on the same ten test images, one for each digit class, so that the comparison is not driven by a single lucky or unlucky example. At each measurement budget M the five methods receive the same measurement matrix A and the same noise vector, and the noise is scaled so that its expected norm remains equal to 0.1 whatever the value of M .

The reconstruction quality is measured as the squared distance between the reconstruction and the original image, divided by the number of pixels:

$$\text{error per pixel} = \frac{\|\hat{x} - x^*\|_2^2}{n}, \quad n = 784. \quad (6.1)$$

This is not the same quantity as the measurement residual $\|AG(\hat{z}) - y\|$ that the recovery algorithm minimizes. The residual decreases as M decreases, simply because fewer constraints have to be satisfied, so it describes how well the optimization converged rather than how good the reconstruction is. For this reason it is used only as a diagnostic, and every number reported below is computed against the ground truth image.

All the reconstructions obtained with a generative prior use a thousand gradient steps and ten random restarts, keeping the restart with the smallest measurement residual.

6.2 BASELINE PERFORMANCE USING LASSO

We started by evaluating the baseline performance using LASSO for reconstruction. LASSO (Least Absolute Shrinkage and Selection Operator) assumes sparsity in the signal and solves the following optimization problem:

$$\hat{x} = \arg \min_x \|y - Ax\|_2^2 + \lambda \|x\|_1, \quad (6.2)$$

where y is the measurement vector, A is the measurement matrix, and λ is a

regularization parameter. We solve it in an orthonormal DCT basis, which is the domain in which MNIST digits are compressible.

From our experiments, we observed that the performance of LASSO is inadequate for this task until a substantial number of measurements is available. Below 100 measurements the reconstruction is dominated by background noise, and a coherent digit only emerges from about 200 measurements onwards. On the other hand LASSO keeps improving as measurements are added, and it is the most accurate method of all once the budget is large, as discussed below.

6.3 COMPARISON OF THE FIVE METHODS

Table 6.1 and figure 6.1 report the error per pixel of every method as the number of measurements grows.

M	LASSO	VAE $k=20$	VAE $k=30$	DCGAN $k=20$	DCGAN $k=30$
10	0.1539	0.0696	0.0782	0.1048	0.0798
25	0.1049	0.0401	0.0250	0.0623	0.0487
50	0.0850	0.0328	0.0153	0.0426	0.0321
75	0.0707	0.0336	0.0111	0.0245	0.0315
100	0.0560	0.0287	0.0158	0.0144	0.0270
200	0.0366	0.0319	0.0071	0.0105	0.0210
300	0.0206	0.0268	0.0067	0.0109	0.0219
400	0.0117	0.0311	0.0067	0.0099	0.0196
500	0.0065	0.0267	0.0068	0.0093	0.0209
750	0.0015	0.0265	0.0067	0.0094	0.0189

Table 6.1: Error per pixel, averaged over ten test digits. The best method at each budget is in bold.

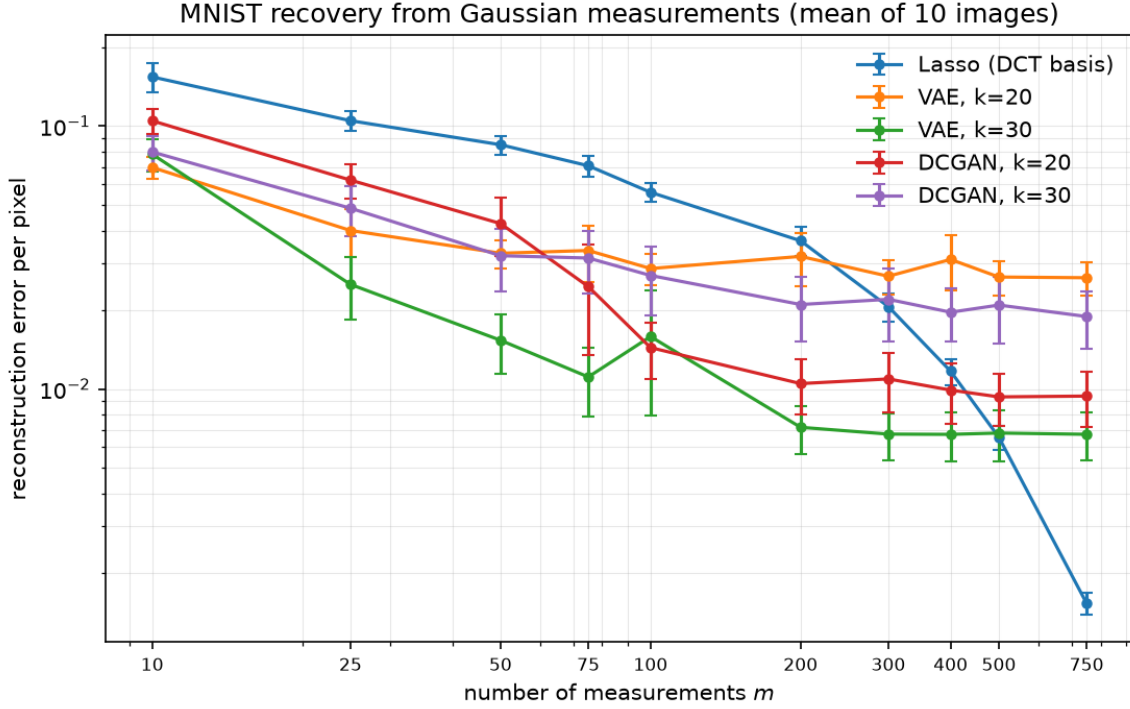


Figure 6.1: Error per pixel as a function of the number of measurements M . Bars are standard errors over the ten test images. Both axes are logarithmic.

Three regimes can be distinguished.

Few measurements. Up to about one hundred measurements the generative priors are far more accurate than LASSO. With 25 measurements the VAE with latent dimension 30 has an error of 0.0250 against 0.1049 for LASSO, that is 4.2 times lower, and at that budget the LASSO reconstruction is not recognizable as a digit at all. This is the regime that motivates compressed sensing, and it is where the choice of the prior matters most.

Sample efficiency. LASSO needs 400 measurements to reach an error of 0.0117. The VAE with latent dimension 30 reaches the same level with 75 measurements, a saving of a factor 5.3, and the DCGAN with latent dimension 20 with 200 measurements, a factor 2. Bora et al. [1] report a saving between 5 and 10 times for their models, so our best model sits at the lower end of the range they describe.

Many measurements. From 500 measurements onwards LASSO becomes

the most accurate method and keeps improving, reaching 0.0015 with 750 measurements against 0.0067 for the best generative model, a reversal by a factor 4.4. This behaviour is expected and is described in [1], where the authors observe that LASSO eventually surpasses the generative approach and that this requires more than 500 measurements. Our experiment reproduces that threshold.

The reason for the reversal is that the reconstruction is constrained to lie in the range of the generator. The distance between a real digit and that range does not depend on the number of measurements, so once enough measurements are available to identify the closest point in the range, additional measurements bring no benefit. Averaging the budgets from 200 upwards, this residual error settles at 0.0068 for the VAE with latent dimension 30, 0.0100 for the DCGAN with latent dimension 20, 0.0204 for the DCGAN with latent dimension 30 and 0.0286 for the VAE with latent dimension 20. These four values are properties of the generators and not of the recovery algorithm.

6.4 RECONSTRUCTION EXAMPLES

Figure 6.2 shows the same digit reconstructed by every method at every budget.

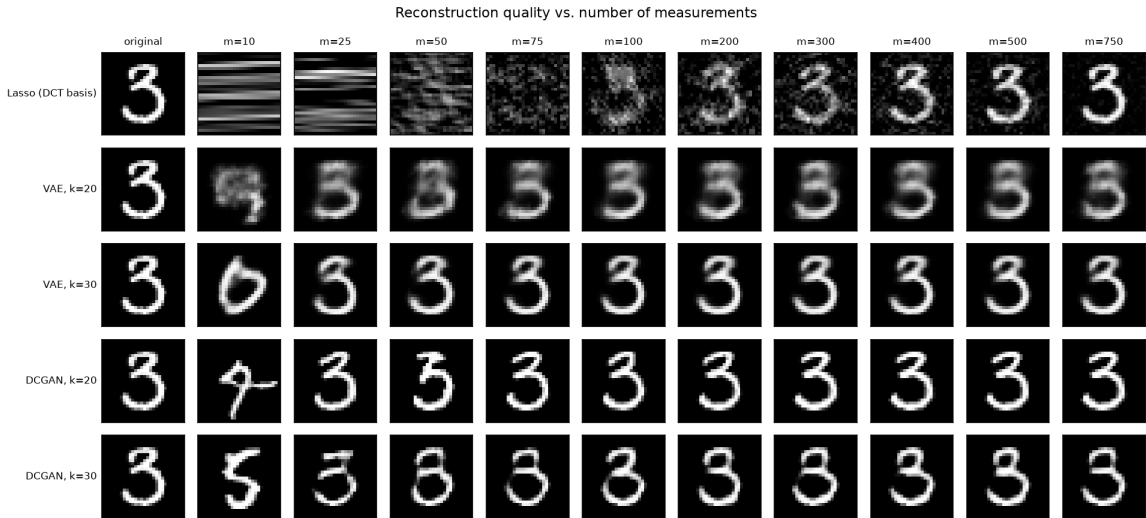


Figure 6.2: One test digit reconstructed by the five methods as the number of measurements increases

The visual comparison confirms the numbers. LASSO returns noise until about 200 measurements. The VAE with latent dimension 20 produces a plau-

sible digit almost immediately, but it never sharpens, and even with 750 measurements it returns a shape halfway between a three and a five. The DCGAN with latent dimension 20 gives a sharp digit from 25 measurements onwards, which is consistent with the fact that an adversarial generator is trained to produce samples that a discriminator accepts, rather than an average of the plausible reconstructions as the VAE does.

6.5 THE LATENT DIMENSION

For the VAE the latent dimension makes a clear difference. The version with $k = 30$ is more accurate than the one with $k = 20$ at every budget from 50 upwards, and the gap ranges between 2.6 and 4.7 standard errors, so it is not attributable to the choice of the test images. A larger latent space describes the digits more faithfully and therefore lowers the error floor discussed above. For the DCGAN the situation is less clear. The version with $k = 20$ is ahead from 75 measurements onwards and the sign of the difference is consistent, but over ten test images the gap never exceeds 1.9 standard errors. We therefore report it as a trend and not as a conclusion, since establishing it would require a larger set of test images.

6.6 COMPUTATIONAL COST

The five methods are not comparable in cost. Reconstructing the ten test images at one budget, with ten restarts and a thousand gradient steps, takes on average 1.4 seconds with LASSO, about 20 seconds with a VAE decoder and about 790 seconds with a DCGAN generator. The factor 38 between the two generative priors is explained by the size of the generator, since the DCGAN generator ends with a transposed convolution producing 512 feature maps at full resolution. The complete experiment reported in this section required 4.5 hours of computation.

This cost is incurred at every reconstruction and not only once during training, because recovery is itself an optimization problem. In an application where measurements are cheap and reconstructions are frequent, this is on its own a decisive argument in favour of LASSO.

7 CONCLUSIONS

This project aimed to assess the performance of compressed sensing using two deep learning models, VAE and DCGAN, on the MNIST dataset. The models' efficacy in image reconstruction from highly compressed measurements was evaluated and compared to a traditional method, LASSO with a DCT basis.

Key Findings:

1. Generative priors win when measurements are scarce:

- With few measurements both models are far more accurate than LASSO. With 25 measurements the VAE with latent dimension 30 has an error per pixel 4.2 times lower, at a budget where the LASSO reconstruction is not recognizable.
- The VAE with latent dimension 30 reaches with 75 measurements the accuracy that LASSO requires 400 measurements to obtain, a saving of a factor 5.3. This is consistent with the range of 5 to 10 reported in [1].

2. The advantage has a ceiling:

- The accuracy of the generative priors stops improving after roughly 200 measurements, because the reconstruction is constrained to the range of the generator and the distance between a real digit and that range does not depend on the number of measurements.
- From 500 measurements onwards LASSO becomes the most accurate method, and with 750 measurements it is 4.4 times more accurate than the best generative model. The same reversal, and the same threshold of about 500 measurements, is described in [1].

3. The VAE outperformed the DCGAN:

- Contrary to what the difference in training cost might suggest, the best model overall is the VAE with latent dimension 30, which is the most accurate of the five methods at seven of the ten budgets we tested.
- The DCGAN produces visually sharper reconstructions, which is expected from an adversarially trained generator, but sharpness and accuracy are not the same thing: a sharp digit of the wrong shape scores worse than a slightly blurred digit of the right one.

4. Cost is part of the comparison:

- Recovery with a DCGAN generator is about 38 times slower than with a VAE decoder, and this cost is paid at every reconstruction, since recovery is itself an optimization problem.
- Given that the VAE is also the more accurate of the two, it is the better choice for this dataset on both counts.

Future Directions:

The results suggest several avenues for further research:

- **Reducing the representation error:** the ceiling described above is a property of the generator, so the natural way to improve the method in the high measurement regime is to train a more expressive generative model rather than to refine the recovery algorithm.
- **Regularizing the latent search:** the objective of [1] includes a term $\lambda \|z\|^2$ that keeps the solution in the region where the prior places most of its mass. We used $\lambda = 0$, and quantifying the effect of that term is a direct extension of this work.
- **Optimizing measurement matrices:** designing or learning measurement matrices adapted to a specific dataset, keeping in mind that the theoretical guarantee relies on the randomness of A and would be traded away.
- **Applications beyond MNIST:** MNIST digits form a simple and low dimensional manifold. Applying the method to more complex datasets would test whether the advantage in the low measurement regime is preserved.

In conclusion, combining a generative prior with compressed sensing is worthwhile exactly when measurements are the scarce resource, which is the situation compressed sensing was designed for. The gain is bounded by how well the generator represents the data, and it is paid for with a reconstruction that is orders of magnitude more expensive than a convex solve. Both facts have to be weighed in applications such as medical imaging, where a single measurement can be slow, costly or harmful to the patient, and where the reconstruction happens once.

REFERENCES

- [1] Ashish Bora, Ajil Jalal, Eric Price, and Alexandros G. Dimakis. Compressed sensing using generative models. In *Proceedings of the 34th International Conference on Machine Learning*, pages 537–546, 2017. arXiv:1703.03208.
- [2] Emmanuel J. Candès, Justin Romberg, and Terence Tao. Robust uncertainty principles: Exact signal reconstruction from highly incomplete frequency information. *IEEE Transactions on Information Theory*, 52(2):489–509, 2006.
- [3] David L. Donoho. Compressed sensing. *IEEE Transactions on Information Theory*, 52(4):1289–1306, 2006.
- [4] Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in Neural Information Processing Systems 27*, pages 2672–2680, 2014.
- [5] KDnuggets. Generative adversarial networks: A hot topic in machine learning. <https://www.kdnuggets.com/2017/01/generative-adversarial-networks-hot-topic-machine-learning.html>, 2017.
- [6] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015. arXiv:1412.6980.
- [7] Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. In *International Conference on Learning Representations*, 2014. arXiv:1312.6114.
- [8] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [9] Alec Radford, Luke Metz, and Soumith Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. In *International Conference on Learning Representations*, 2016. arXiv:1511.06434.

- [10] Rushikesh Shende. Autoencoders, variational autoencoders (vae) and beta-vae. <https://medium.com/@rushikesh.shende/autoencoders-variational-autoencoders-vae-and-beta-vae-ceba9998773d>, 2023.
- [11] Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society, Series B*, 58(1):267–288, 1996.